



Licenciatura em Engenharia Informática e de Computadores

FOCCACIA API

Trabalho Prático

Trabalho realizado por:

Nome: Tiago Gomes N°51854

Nome: Diogo Teixeira N°50506

Nome: Miguel Pinto N°51557

Grupo: 10

Turma: 33D

Docente: Prof. Luís Falcão

Introdução à Programação na Web

2024/2025 Inverno

3 de janeiro 2025

Índice:

1. Introdução	3
2. Estrutura da aplicação	4
2.1. Visão Geral do Sistema	4
2.2. Componentes do Cliente.....	6
2.3. Componentes do Servidor	7
2.4. Fluxo de Dados e Comunicação.....	10
2.5. Diagrama de Arquitetura	12
3. Design do Armazenamento de Dados	14
4. Mapeamento entre Elasticsearch e o modelo de objetos	16
5. Documentação da API do Servidor	18
6. Instruções para executar a aplicação e os testes.....	21
7. Conclusão	22

1. Introdução

O projeto FOCCACIA (FOotball Complete Clubs API and Chelas Internet Application) visa criar uma aplicação web robusta e flexível para a gestão de equipas e grupos desportivos, utilizando tecnologias modernas e princípios de desenvolvimento web, como APIs RESTful e bases de dados NoSQL. O objetivo principal é oferecer aos utilizadores a possibilidade de pesquisar e gerir equipas de futebol, associar-se a grupos, adicionar ou remover equipas desses grupos, e visualizar informações detalhadas sobre as equipas e ligas, tudo através de uma interface web intuitiva.

Este trabalho foi desenvolvido ao longo de três partes, com o foco em incrementos sucessivos nas funcionalidades da aplicação, seguindo uma metodologia de refatoração contínua e aplicação de boas práticas de programação. As 3 partes do desenvolvimento do trabalho são as seguintes:

- **1ª Parte:** A primeira fase envolve a criação de uma API RESTful básica, permitindo operações como a pesquisa de equipas, gestão de grupos (criação, edição, remoção e visualização) e manipulação de dados em memória. O servidor foi desenvolvido com Node.js, utilizando o módulo express para lidar com as requisições HTTP e a integração com a API Football para a obtenção de dados externos.
- **2ª Parte:** A segunda fase introduz melhorias significativas, incluindo a migração do armazenamento de dados para Elasticsearch, o que melhora a escalabilidade e a eficiência na manipulação dos dados da aplicação. Além disso, foi criada uma interface web que permite aos utilizadores interagir com a aplicação sem precisar conhecer detalhes técnicos, como IDs de equipas ou grupos. A interface foi desenvolvida usando HTML, CSS, Handlebars e Bootstrap, garantindo uma apresentação visual clara e agradável.
- **3ª Parte:** A fase final focou-se na implementação de funcionalidades de autenticação de utilizadores, através do módulo Passport, garantindo que a gestão de grupos e equipas fosse restrita a utilizadores autenticados e que cada grupo fosse privado e apenas manipulável pelo seu proprietário. A aplicação passou a ter um nível de segurança adicional, permitindo a personalização da gestão de grupos conforme o utilizador.

Os objetivos principais deste trabalho foram garantir a integração correta entre a aplicação web e a API, implementar funcionalidades que permitam uma gestão flexível de grupos e equipas, e assegurar que a aplicação seja acessível e segura para os utilizadores. Ao longo das três fases, foram implementadas melhorias tanto na lógica de funcionamento da API quanto na interface do utilizador, com a preocupação de manter a aplicação escalável, eficiente e fácil de usar.

2. Estrutura da aplicação

A aplicação FOCCACIA foi desenvolvida seguindo uma arquitetura modular e organizada, garantindo a separação de responsabilidades entre os diferentes componentes. Essa abordagem permite maior clareza no desenvolvimento, facilidade de manutenção e escalabilidade do sistema. Este tópico descreve a estrutura da aplicação, detalha os principais módulos do cliente e servidor, e apresenta como eles se relacionam para implementar as funcionalidades propostas. Adicionalmente, é fornecido um diagrama para ilustrar as interações entre os componentes do sistema.

2.1. Visão Geral do Sistema

A aplicação FOCCACIA foi desenvolvida utilizando uma arquitetura modular baseada na separação de responsabilidades entre o **cliente** (front-end) e o **servidor** (back-end). Essa abordagem não apenas simplifica a manutenção e o desenvolvimento do sistema, mas também permite uma maior escalabilidade e clareza na organização do código.

Cliente (Front-end):

O cliente é responsável por oferecer uma interface gráfica para os utilizadores interagirem com a aplicação. Ele atua como a camada de apresentação, facilitando a comunicação com o servidor e garantindo que todas as funcionalidades sejam acessíveis de forma intuitiva.

O cliente, gerido pelo módulo **focaccia-web-site.mjs**, é responsável por renderizar páginas HTML no lado do servidor e enviar request's HTTP à API RESTful para operações como criar grupos ou adicionar equipas. Utiliza HTML, CSS, Handlebars e Bootstrap, garantindo uma experiência de utilizador intuitiva e eficiente.

Servidor (Back-end):

O servidor é o núcleo da aplicação, responsável por gerenciar a lógica de negócio, processar as requisições do cliente, integrar-se com serviços externos (API Football) e armazenar dados no ElasticSearch.

O servidor, estruturado de forma modular, centraliza a lógica de negócio, processa requisições do cliente e interage com serviços externos (API Football e ElasticSearch). Os módulos principais incluem:

- **foccacia-server.mjs**
- **foccacia-web-api.mjs**
- **foccacia-service.mjs**
- **foccacia-fetch.mjs**
- **foccacia-data-es.mjs & users-data-elastic.mjs**

2.2. Componentes do Cliente

O módulo **focaccia-web-site.mjs** é responsável por gerir a camada de apresentação da aplicação FOCCACIA, implementando as rotas HTTP necessárias para a interface do utilizador. Este módulo utiliza dados fornecidos pelos serviços backend para renderizar páginas dinâmicas e responder às interações do utilizador. As principais responsabilidades incluem:

- **Renderização de Páginas HTML Dinâmicas:**
 - Fornece páginas como a lista de grupos (/groups) e os detalhes de um grupo específico (/groups/:groupId), preenchidas com dados em tempo de execução.
- **Gerenciamento de Requisições HTTP:**
 - Implementa rotas GET para exibir recursos e POST para criação ou modificação de dados, como grupos e equipas.
- **Tratamento de Erros:**
 - Inclui mecanismos para capturar e comunicar falhas, garantindo que os utilizadores recebam feedback apropriado em caso de problemas.

Tecnologias Utilizadas

- **Handlebars:**
 - Utilizado para renderizar templates HTML dinâmicos no servidor. Este sistema de templates permite a geração de páginas reutilizáveis, preenchidas com dados provenientes do backend, como a lista de grupos.
- **CSS e Bootstrap:**
 - CSS é usado para o estilo básico das páginas, enquanto o Bootstrap garante uma interface responsiva e esteticamente consistente. Elementos como botões, tabelas e formulários são estilizados com classes do Bootstrap, facilitando o desenvolvimento e a manutenção do design.
- **Express.js:**
 - Este módulo define e gere rotas HTTP que conectam as interações do utilizador à lógica do backend. A integração com Handlebars permite a renderização de templates de forma eficiente.

2.3. Componentes do Servidor

O servidor da aplicação FOCCACIA foi projetado de forma modular, dividindo responsabilidades entre diferentes ficheiros/módulos. Cada componente tem um papel específico, desde a receção de requests até a integração com serviços externos. De seguida, estão descritos os principais módulos do servidor:

2.3.1. foccacia-server.mjs

Papel:

Este módulo é o ponto de entrada do servidor. Ele inicializa o servidor HTTP, configura middlewares e regista as rotas da aplicação.

Principais Funções:

1. Inicializa o servidor utilizando o framework **Express**.
2. Regista as rotas definidas no módulo **foccacia-web-api.mjs**.
3. Carrega configurações gerais e middlewares, como:
 - Middleware para processar requisições no formato JSON.
 - Integração com o módulo de autenticação **Passport**.

Fluxo Geral: Quando o servidor é iniciado, este módulo prepara a aplicação para receber e processar requisições HTTP.

2.3.2. foccacia-api.mjs

Papel:

Este módulo define os endpoints RESTful da API. Cada endpoint é responsável por tratar uma funcionalidade específica, como criar grupos ou buscar equipas.

Principais Funções:

1. Define rotas HTTP para operações do sistema, como:
 - **GET /groups**: Lista todos os grupos.
 - **POST /groups**: Cria novo grupo.
 - **GET /groups/:groupId/teams**: Busca equipas associadas a um grupo.
2. Encapsula handlers com a função auxiliar handlerWrapper, que assegura o tratamento centralizado de erros e validações.
3. Chama o módulo **foccacia-services.mjs** para delegar a lógica de negócio.

Fluxo Geral: O módulo recebe requisições do cliente, valida os dados e delega as operações aos serviços correspondentes.

2.3.3. foccacia-service.mjs

Papel:

Este módulo implementa a lógica de negócio da aplicação. Ele atua como uma ponte entre os endpoints e os módulos que interagem com os dados (ElasticSearch ou API Football).

Principais Funções:

1. Manipulação de Grupos:

- Criação, atualização e exclusão de grupos.
- Associação de equipas a grupos.

2. Integração com ElasticSearch:

- Garante que dados persistentes, como grupos e equipas, sejam armazenados ou recuperados do ElasticSearch.

3. Chamada à API Football:

- Quando necessário, utiliza o módulo **foccacia-fetch.mjs** para obter informações externas.

2.3.4. foccacia-fetch.mjs

Papel:

Este módulo é responsável por interagir com a API Football, obtendo informações como equipas, ligas e estádios.

Principais Funções:

1. **fetchIdByName(teamName):** Busca o ID de uma equipa pelo nome.
2. **fetchTeam(teamID):** Obtém detalhes sobre uma equipa específica.
3. **fetchLeagues(teamID, seasonYear):** Lista as ligas associadas a uma equipa numa temporada específica.
4. **Cache Persistente:** Utiliza um sistema de cache para reduzir o número de chamadas à API externa, armazenando os resultados localmente no ficheiro data.json.

Fluxo Geral: O módulo recebe parâmetros do **foccacia-services.mjs**, faz chamadas à API Football e retorna os resultados formatados.

2.3.5. users-data-elastic.mjs & foccacia-data-es.mjs

Papel:

Este módulo gere a interação com o ElasticSearch, que é utilizado como sistema de armazenamento persistente para dados da aplicação.

Principais Funções:

1. **Armazenamento de Dados:** Salva informações sobre grupos, equipes e outros dados no ElasticSearch.
2. **Consulta de Dados:** Executa buscas otimizadas para recuperar informações solicitadas pelo cliente.
3. **Integração Transparente:** Abstrai a complexidade do ElasticSearch, fornecendo uma interface simples para o foccacia-services.mjs.

2.4. Fluxo de Dados e Comunicação

Fluxo Entre Cliente, Servidor, Elasticsearch e API Football:

1. Do Cliente para o Servidor:

- O cliente envia uma requisição HTTP para realizar operações, como criar um grupo, pesquisar equipas ou consultar detalhes de uma liga.
- A requisição é encaminhada para o servidor, onde os endpoints definidos no **foccacia-web-api.mjs** processam a operação.

2. Do Servidor para o Elasticsearch:

- Quando a operação envolve manipulação de dados persistentes (ex.: armazenar grupos ou consultar equipas associadas a um grupo), o servidor interage com o Elasticsearch através do módulo responsável.
- O Elasticsearch processa a consulta ou armazena os dados e retorna uma resposta ao servidor.

3. Do Servidor para a API Football:

- Quando os dados solicitados não estão disponíveis localmente, o servidor utiliza o módulo **foccacia-fetch.mjs** para consultar a API Football.
- Exemplo: Ao pesquisar uma equipa, a API Football fornece informações como o nome, estádio e liga.

4. Do Servidor para o Cliente: Após processar a requisição, o servidor retorna a resposta ao cliente, podendo ser:

- **JSON:** Para APIs RESTful.
- **Página Renderizada:** Para a interface web.

Tratamento de Requisições e Respostas:

1. Receção e Encaminhamento:

Cada requisição é encaminhada ao handler correspondente no foccacia-web-api.mjs. Por exemplo:

- **GET /groups:** Lista todos os grupos.
- **POST /groups:** Cria um grupo.

2. Validação e Processamento:

O handler valida os dados recebidos e chama as funções no groupsService para executar a lógica necessária.

Funções auxiliares como handlerWrapper asseguram o tratamento centralizado de erros.

3. Interação com Outros Sistemas:

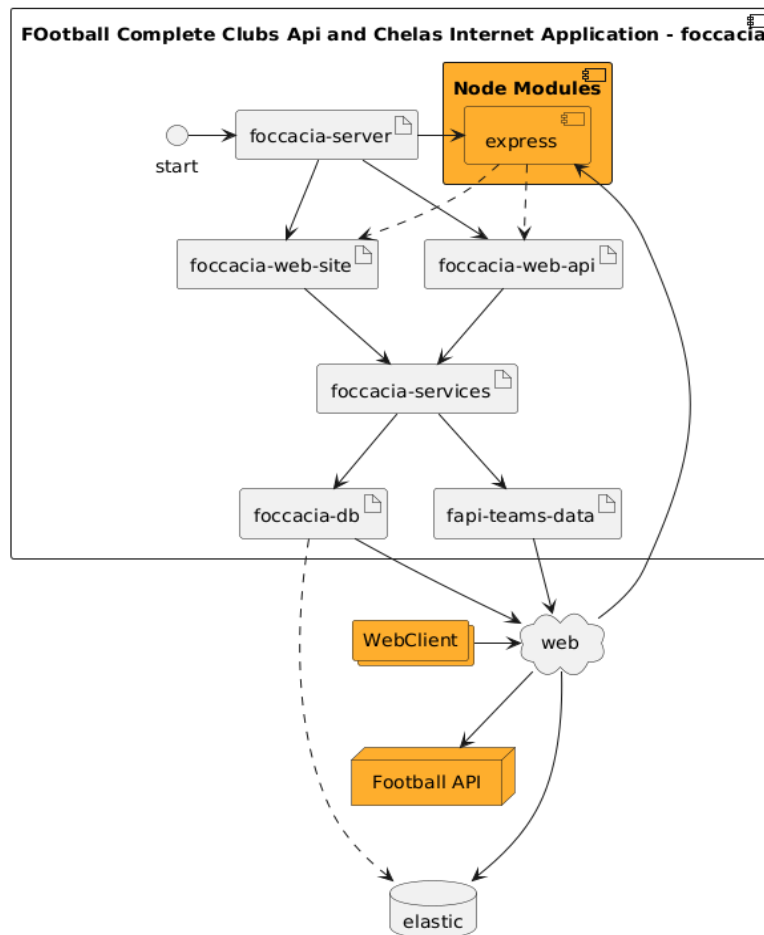
- **ElasticSearch:** Para consultas e armazenamento local de dados persistentes.
- **API Football:** Para obter informações externas, como equipas e ligas.

4. Resposta ao Cliente:

Após o processamento, os dados são enviados ao cliente:

- Se a operação for bem-sucedida, retorna-se os dados solicitados ou redireciona-se o utilizador.
- Em caso de erro, uma mensagem clara é retornada (ex.: 400 Bad Request ou 500 Internal Server Error).

2.5. Diagrama de Arquitetura do FOCCACIA



Descrição do Diagrama

1. Camada Inicial:

- **foccacia-server**: Representa o ponto de entrada do servidor, onde as rotas são configuradas e a aplicação é inicializada. Ele utiliza o módulo express para lidar com as requisições HTTP e conectar os módulos subsequentes.

2. Camada de Interface Web:

- **foccacia-web-site**: gerência as requisições provenientes do cliente web e renderiza páginas dinâmicas utilizando Handlebars e Bootstrap.
- **foccacia-web-api**: Define os endpoints da API RESTful, conectando o cliente e os serviços.

3. Camada de Serviços:

- **foccacia-service**: Contém a lógica de negócio da aplicação, processando dados antes de serem enviados ao cliente ou armazenados no banco de dados.

4. Camada de Dados:

- **Foccacia-fetch**: Realiza chamadas à API Football para obter dados externos, como informações de equipas, ligas e estádios.

5. Fontes de Dados Externas:

- **ElasticSearch**: Gerencia o armazenamento e recuperação de dados estruturados.
- **Football API**: Fonte de dados externos utilizada para enriquecer as informações armazenadas no sistema.

A imagem foi retirada do enunciado fornecido pelo professor, alguns nomes de ficheiros não são equivalentes aos nomes utilizados no nosso trabalho, apesar da função dos mesmos ser a mesma.

3. Design do Armazenamento de Dados

O armazenamento de dados no sistema FOCCACIA utiliza o Elasticsearch para gerenciar informações específicas da aplicação, como grupos, equipas e ligas. A seguir, detalhamos os índices criados, as propriedades dos documentos e as relações entre eles.

Índices criados no Elasticsearch:

- **groups:**
 - Armazena informações sobre grupos criados pelos utilizadores.
 - Contendo o nome, descrição e a lista de equipas associadas.
- **teams:**
 - Contém detalhes sobre equipas, incluindo nome, estádio e liga.
 - Utilizado para armazenar equipas adicionadas a grupos ou pesquisadas através da Football API.
- **leagues:**
 - Regista informações sobre ligas em que as equipas participam, com detalhes como nome, país e temporadas disponíveis.

Propriedades dos Documentos:

Groups:

```
{
  "groupId": "12345",
  "name": "Fãs de Futebol",
  "description": "Grupo dedicado aos amantes do futebol",
  "teams": [
    {
      "teamId": "211",
      "name": "Benfica",
      "stadium": "Estádio da Luz",
      "league": "Primeira Liga",
      "season": "2023"
    }
  ],
  "owner": "user123"
}
```

groupId: Identificador único do grupo.

name e description: Nome e descrição do grupo.

teams: Lista de equipas associadas ao grupo, com detalhes básicos.

owner: Identificador do utilizador proprietário do grupo.

Team:

```
{
  "teamId": "211",
  "name": "Benfica",
  "stadium": {
    "name": "Estádio da Luz",
    "capacity": 65647,
    "city": "Lisboa"
  },
  "league": {
    "id": "94",
    "name": "Primeira Liga",
    "country": "Portugal"
  },
  "logo": "https://media.api-sports.io/football/teams/211.png"
}
```

teamId: Identificador único da equipa.

name: Nome da equipa.

stadium: Detalhes sobre o estádio.

league: Informações da liga associada.

logo: URL do logotipo da equipa.

League:

```
{
  "leagueId": "94",
  "name": "Primeira Liga",
  "country": "Portugal",
  "seasons": [
    {
      "year": 2023,
      "start": "2023-08-12",
      "end": "2024-05-19"
    }
  ]
}
```

leagueId: Identificador único da liga.

name e country: Nome e país da liga.

seasons: Lista de temporadas com datas de início e fim.

Relações entre Documentos:

- **Entre groups e teams:**
 - O índice groups referencia documentos do índice teams por meio da propriedade teamId. Essa relação permite listar as equipes associadas a cada grupo.
- **Entre teams e leagues:**
 - O índice teams referencia o índice leagues, associando equipes às ligas em que participam.

A utilização do Elasticsearch no projeto FOCCACIA proporcionou uma estrutura escalável, flexível e eficiente para o armazenamento e recuperação de dados. A modelagem clara dos índices e documentos permite lidar com grandes volumes de informações, acomodar futuras expansões e integrações, e realizar consultas rápidas, essenciais para oferecer uma experiência fluida aos utilizadores. Este design fortalece a base técnica da aplicação, garantindo robustez e desempenho ao longo do tempo.

4. Mapeamento entre Elasticsearch e o modelo de objetos

O mapeamento entre o Elasticsearch e o modelo de objetos da aplicação FOCCACIA é crucial para garantir que os dados armazenados sejam interpretados e utilizados corretamente pelos módulos do servidor. De seguida, é explicado como este mapeamento é implementado, são descritas as funções responsáveis e apresentados exemplos de mapeamentos.

1. Transformação de Dados do Elasticsearch em Objetos da Aplicação:

Os dados armazenados no Elasticsearch são organizados em índices (por exemplo, groups, teams) e representados em formato JSON. Quando recuperados, esses dados são transformados em objetos JavaScript que a aplicação pode manipular.

2. Funções de Mapeamento Implementadas no Servidor

As funções de mapeamento estão geralmente localizadas no módulo responsável por interagir com o Elasticsearch (ex.: foccacia-data-elastic.mjs) ou em serviços como foccacia-services.mjs. Elas garantem que os dados sejam convertidos de e para o formato esperado pela aplicação.

2.1. Mapeamento de Grupos

- **Função:** Recupera grupos armazenados no Elasticsearch e os transforma em objetos manipuláveis no servidor.

2.2. Mapeamento de Equipas

- **Função:** Busca informações de equipas armazenadas e retorna objetos prontos para renderização.

2.3. Mapeamento de Ligas

- **Função:** Lista ligas associadas a uma equipa específica.

Benefícios do Mapeamento

1. **Consistência:**

Garante que os dados manipulados no servidor estão no formato esperado, independentemente de como estão armazenados no Elasticsearch.

2. **Flexibilidade:**

Permite ajustar a estrutura dos objetos sem alterar a forma como os dados são armazenados no Elasticsearch.

3. **Eficiência:**

Reduz a complexidade na interação com o Elasticsearch, centralizando a lógica de transformação em funções reutilizáveis.

5. Documentação da API do Servidor

Método	Caminho(url)	Descrição
GET	/groups	Lista todos os grupos criados pelo utilizador autenticado.
GET	/groups/:groupId	Obtém os detalhes de um grupo específico.
POST	/groups	Cria um novo grupo.
POST	/groups/:groupId	Atualiza os detalhes de um grupo específico.
POST	/groups/:groupId/delete	Remove um grupo específico.
GET	/groups/:groupId/teams	Procura equipas associadas a um grupo ou pesquisa equipas na API Football.
POST	/groups/:groupId/modify-team	Adiciona ou remove equipas de um grupo
POST	/login/register-action	Regista um novo utilizador.
POST	/login-action	Realiza o login do utilizador.
GET	/site/index	Exibe a página inicial da aplicação web.
GET	/login/register	Exibe a página de registo para criar uma nova conta.

Explicação do Código que programa as rotas:

1. Responsabilidade

O ficheiro **foccacia-web-api.mjs** é responsável por definir os endpoints HTTP da aplicação. Ele conecta os requests dos utilizadores aos serviços da aplicação e implementa handlers que gerem diferentes operações, como criação de grupos, pesquisa de equipas, login, etc.

2. Estrutura Geral:

- **Rotas Definidas com `express.router`:** Cada endpoint é implementado como uma rota utilizando o framework Express.
- **Funções auxiliares:**
 - `setUserToken`: Configura o token do utilizador autenticado.
 - `handlerWrapper`: Um wrapper que encapsula a lógica para o tratamento de erros.

3. Principais Funções:

- **`getGroups`**: Procura e renderiza a lista de grupos do utilizador em questão.
- **`getGroup`**: Obtém os detalhes de um grupo específico.
- **`createGroup`**: Cria um grupo.
- **`updateGroup`**: Atualiza informações de um grupo.
- **`deleteGroup`**: Remove um grupo existente.
- **`deleteTeam`**: Remove uma equipa de dentro de um certo grupo.
- **`searchTeam`**: Pesquisa equipas associadas a um grupo ou na API externa.
- **Autenticação com Passport**: Inclui endpoints para login e registo de utilizadores utilizando o Passport.

A autenticação na API FOCCACIA é gerida utilizando o framework **Passport**, garantindo que apenas utilizadores autenticados possam aceder a determinadas funcionalidades. Abaixo, é descrito o funcionamento da autenticação e a gestão de tokens para proteger os endpoints da API:

1. Utilização do Passport:

O **Passport** é o middleware responsável pela autenticação de utilizadores. Ele é configurado para utilizar a estratégia local, que autentica os utilizadores com base nas credenciais fornecidas que neste caso são o username (nome de utilizador) e a password (palavra-passe).

Configuração do Passport:

A configuração é realizada no ficheiro `passport-config.mjs`, onde:

- São definidas as estratégias de autenticação.
- É implementada a verificação de credenciais, utilizando as funções do `groupsService` para validar utilizadores na base de dados.
- O Passport serializa e desserializa as sessões do utilizador para facilitar a autenticação persistente.

2. Token do Utilizador:

O token é utilizado para identificar e autorizar o utilizador em cada requisição após a autenticação.

- **Definição do Token:**

Após o utilizador fazer login, o Passport associa um token único (gerado na criação do utilizador) ao objeto do utilizador em `req.user`.

- **Associação do Token às Requisições:**

A função auxiliar `setUserToken` verifica se o utilizador está autenticado e, caso esteja, associa o token ao objeto da requisição (`req.token`), permitindo que os serviços utilizem este token para validar permissões ou identificar dados do utilizador.

3. Proteção de Endpoints:

Apenas utilizadores autenticados podem aceder a endpoints sensíveis, como criar, editar ou excluir grupos. Isso é garantido por meio do Passport e da validação do token associado à sessão do utilizador.

Exemplo de Endpoints Protegidos:

- **Criar Grupo (POST /groups):**

O token do utilizador é verificado antes de permitir a criação do grupo.

- **Editar Grupo (POST /groups/:groupId):**

A operação é autorizada apenas se o utilizador autenticado possuir permissões para o grupo.

- **Excluir Grupo (POST /groups/:groupId/delete):**

O token é necessário para verificar se o utilizador é o proprietário do grupo.

Fluxo de Verificação do Token, passo a passo:

1. O utilizador faz login e recebe o token associado à sessão.
2. Para cada requisição, a função `setUserToken` verifica se o token está presente e é válido.
3. Endpoints protegidos utilizam este token para garantir que o utilizador tem permissão para realizar a ação.

6. Instruções para executar a aplicação

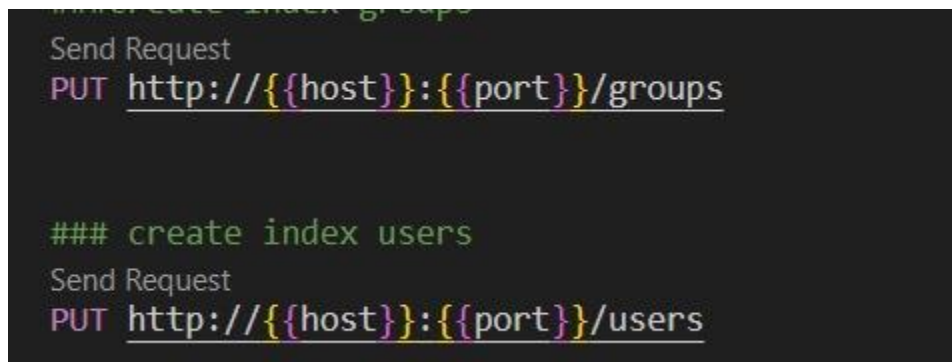
- Instalar o Elastic Search:

Para correr vá à paste bin e clique no primeiro ficheiro .bat, deves deixar isso aberto no decorrer da utilização

Já no terminal, deverá executar todos estes comandos:

```
-npm install express;  
-npm install -g nodemon;  
-powershell -Command "Set-ExecutionPolicy Bypass -Scope Process;  
-npm install cors;  
-npm install hbs;  
-nodemon focaccia-server.mjs"
```

Depois de instalar tudo e ter o código a correr, deverá realizar estes dois requests que se encontram no ficheiro elastic-search.http:



```
### create index groups  
Send Request  
PUT http://{{host}}:{{port}}/groups  
  
### create index users  
Send Request  
PUT http://{{host}}:{{port}}/users
```

De seguida, depois do servidor estar ligado e funcional, basta abrir o seu browser e pesquisar: <http://localhost:3310/site/login>

Se tudo correr bem, o site irá abrir e aparecerá a haba onde se pode registar ou fazer login no nosso site.

7. Conclusão

O desenvolvimento do projeto FOCCACIA ao longo do semestre representou uma experiência enriquecedora, tanto no que diz respeito à aplicação prática de conceitos teóricos, quanto no aprimoramento de competências técnicas e de trabalho em equipa. Através de três fases distintas, foi possível construir uma aplicação completa que atende aos requisitos funcionais e não funcionais propostos, integrando tecnologias modernas e boas práticas de desenvolvimento.

Na **primeira fase**, estabelecemos as bases do projeto, com o desenvolvimento de uma API RESTful funcional que permitia a gestão de grupos e equipas, além da integração inicial com a API Football para pesquisa de dados. Essa etapa foi fundamental para definir a arquitetura do sistema e garantir a modularidade e escalabilidade do código.

Na **segunda fase**, avançámos com a migração para o Elasticsearch, melhorando a eficiência no armazenamento e recuperação de dados. Além disso, desenvolvemos a interface web, que trouxe maior usabilidade ao sistema, eliminando a necessidade de interação direta com IDs e permitindo que os utilizadores acessem às funcionalidades de forma intuitiva e simples.

Por fim, na **terceira fase**, consolidámos a aplicação com a implementação de autenticação de utilizadores, utilizando o módulo Passport. Esta funcionalidade assegurou a privacidade e segurança dos dados, permitindo que apenas utilizadores autenticados pudessem gerir os seus grupos e equipas. Além disso, reforçámos a integração entre a API e a interface, garantindo que todas as operações fossem acessíveis tanto pelo cliente quanto pelo servidor.

Os principais desafios enfrentados ao longo do projeto incluíram a adaptação às limitações da API Football, a modelagem dos dados em Elasticsearch e a implementação de uma interface web que fosse simultaneamente funcional e amigável. No entanto, esses desafios foram superados com a aplicação de boas práticas de programação, como modularidade, testes unitários e de integração, e uma abordagem incremental para o desenvolvimento das funcionalidades.

No final, o FOCCACIA destaca-se como uma aplicação robusta e completa, que combina eficiência no processamento de dados, segurança e facilidade de uso. Este projeto proporcionou uma oportunidade valiosa para consolidar conhecimentos em tecnologias como Node.js, Elasticsearch, e autenticação de utilizadores, além de promover a experiência prática em desenvolvimento web e APIs RESTful.

Em suma, o projeto FOCCACIA alcançou com sucesso os objetivos propostos, oferecendo uma aplicação que cumpre todos os requisitos especificados e serve como um exemplo prático de desenvolvimento de sistemas modernos e escaláveis.